

# Rapport de feedback développeur

SecondWave · équipe BFT-PARIS-26

**Track 2** — Lending Protocol    **Flavour** Loaded (Permissioned Domains + Credentials)

**Environnement** Devnet public, wss://s.devnet.ripple.net:51233 · rippled 3.4.0-rc5 (LendingProtocolV1\_1, BatchV1\_1, fixCleanup3\_4\_0 actifs)

**Libs** xrpl.js@5.2.0-beta.1 · ripple-binary-codec@2.11.0 · Node ≥ 20

**Le projet** — un marché secondaire de parts de vault verrouillées. Pendant la phase Investment d'un vault *closed-ended*, `VaultWithdraw` est bloqué : un déposant qui doit sortir n'a aucune issue. Les parts étant un MPT, on les lui fait vendre à un autre membre du fonds, avec un règlement atomique et une analyse du risque à l'achat. Environ 420 cas exécutés sur la chaîne ; chaque point est rejouable par un script du dépôt. Versions longues : `probes/FRICTIONS.md` et `probes-marche/FRICTIONS.md`.

## 1 · Un Batch ne dit pas s'il a produit un effet

catégorie UX · lib rippled 3.4.0-rc5

### Le problème

Précisons d'abord ce qui **n'est pas** le problème : `tesSUCCESS` sur un `tfAllOrNothing` dont rien n'a appliqué est défendable — le drapeau promet « tout ou rien », et le protocole a fait respecter l'invariant.

Le problème est que **les deux issues sont indiscernables**. Réponse complète du serveur sur un Batch qui n'a rien fait :

```
accepted: true | applied: true | engine_result: tesSUCCESS
engine_result_message: "The transaction was applied."
meta.AffectedNodes: [ 1 noeud ]    <- Balance -150 drops, Sequence +1
```

Quatre champs qui disent « ça a marché », dont un `applied: true` alors que rien n'a été appliqué. Le seul nœud de métadonnée est la ponction de frais. Et `tx_json` restitue les `RawTransactions` — c'est-à-dire **ce qu'on a demandé, pas ce qui s'est passé**.

Sur huit Batch rejoués depuis le ledger, cinq ayant déplacé des parts et trois n'ayant rien fait, **une seule combinaison apparaît** : `tesSUCCESS` / 1 nœud. XLS-56 spécifie pourtant un `BatchExecutions` dans la métadonnée, exactement pour distinguer les deux cas. Il est absent.

### Ce qu'on a vécu

On a passé une demi-journée à croire notre code cassé. L'échange renvoyait `tesSUCCESS`, puis les soldes ne bougeaient pas — mais pas toujours. Ce n'est pas aléatoire : chaque jambe est évaluée sur ses propres préconditions, et une seule qui échoue annule tout. Quatre causes distinctes chez nous, toutes invisibles depuis la réponse :

| Cause réelle de l'échec                               | Code que la jambe aurait rendu seule |
|-------------------------------------------------------|--------------------------------------|
| l'acheteur ne peut pas financer le prix               | <code>tecUNFUNDED_PAYMENT</code>     |
| l'acheteur n'est pas membre du domaine                | <code>tecNO_AUTH</code>              |
| l'acheteur n'a pas fait <code>MPTokenAuthorize</code> | <code>tecNO_AUTH</code>              |
| séquence interne décalée                              | <code>terPRE_SEQ</code>              |

Aggravant : **simulate répond notImpl sur un Batch** — ni pré-jouer, ni constater. Il a fallu deux couches maison : retrouver les jambes par `account_tx` sur la plage `[ledgerIndex, ledgerIndex]`, puis comparer les soldes. Environ 150 lignes pour répondre à « est-ce que mon échange a eu lieu ? ».

### Ce qu'on propose

1. **Livrer le BatchExecutions que la spec annonce** — au minimum, par jambe : son index, son hash et son code de résultat.
2. À défaut, **le strict minimum utile** : l'index de la première jambe en échec et son code. Un entier et un code changeraient tout.
3. **Ne pas écrire applied: true** quand aucune jambe n'a appliqué.
4. **Faire fonctionner simulate sur un Batch** : c'est la transaction la plus difficile à construire et la seule qu'on ne peut pas pré-vérifier.

Le moment est favorable : l'amendement est encore en `BatchV1_1` sur une *release candidate*. Changer la métadonnée après activation coûtera bien plus cher.

## 2 · Trois drapeaux de Batch sur quatre livrent l'actif sans encaisser le prix

catégorie UX · lib rippled 3.4.0-rc5

### Le problème

Le même échange — parts contre XRP, jambe du prix volontairement impayable — rejoué sous chaque drapeau :

| Drapeau                                                 | Annoncé                 | Parts déplacées | Prix payé |
|---------------------------------------------------------|-------------------------|-----------------|-----------|
| <code>tfAllOrNothing</code>                             | <code>tesSUCCESS</code> | 0               | 0         |
| <code>tfOnlyOne · tfUntilFailure · tfIndependent</code> | <code>tesSUCCESS</code> | 300 000         | 0         |

C'est cohérent avec la sémantique de chaque mode. Mais appliqué à un échange de valeur, trois modes sur quatre font **livrer sans encaisser** — et les quatre annoncent le succès.

### Ce qu'on a vécu

On a découvert ça en tabulant les quatre drapeaux par curiosité, pas en cherchant un bug. L'oubli est impossible — **Flags**: 0 est refusé par `temINVALID_FLAG`, et le SDK refuse même de co-signer un **Batch** sans le champ. Le risque est donc **le mauvais choix**, fait en connaissance de cause. Combiné au point 1, il est **indétectable** : même code de retour, même métadonnée, et l'actif est parti.

### Ce qu'on propose

Une mise en garde explicite dans XLS-56 pour les échanges de valeur, et un helper de swap dans les SDK qui impose `tfAllOrNothing`. Le drapeau qui protège devrait être celui qu'on obtient sans y penser.

## 3 · Aucune primitive de lecture côté XLS-66

catégorie `missing primitive` · lib `rippled 3.4.0-rc5`

### Le problème

`vault_info` existe. **Ni `loan_info`, ni `loan_broker_info`, ni la liste des détenteurs de parts**. Reconstituer l'état d'un vault impose une traversée à trois niveaux, chacun sur un pseudo-compte différent :

```

| vault_info(vaultId)
|   -> account_objects(pseudo-compte du vault,      type: 'loan_broker')
|   -> account_objects(pseudo-compte du courtier, type: 'loan')
```

Et pour savoir qui détient les parts, aucune commande : il faut **rejouer tout l'historique** du pseudo-compte via `account_tx` et reconstruire les soldes depuis la métadonnée de chaque transaction.

### Ce qu'on a vécu

Notre produit est une aide à la décision : il doit afficher l'état d'un vault avant qu'on achète une position. **Six appels RPC** pour un vault à 3 courtiers et 3 prêts, plus un parcours paginé complet pour les détenteurs. C'est la partie du code qui nous a demandé le plus de travail, et elle ne produit aucune valeur métier — elle reconstitue ce que le serveur sait déjà.

Un point positif : le jeu est **auto-vérifiant** — la somme reconstruite retombe exactement sur `shares.OutstandingAmount`.

### Ce qu'on propose

`loan_broker_info` et `loan_info` sur le modèle de `vault_info`, et surtout une commande qui liste les détenteurs d'une émission MPT. Le jeu d'historique devrait être un cas limite, pas le chemin normal.

## 4 · `LendingProtocolV1_1` est actif et n'est documenté nulle part

catégorie `documentation/tutorials` · lib `rippled 3.4.0-rc5`

### Le problème

`VaultKind`, `SubscriptionDate`, `RedemptionDate` et `LEVersion` existent sur le ledger et sont connus du codec. **Aucune spécification publiée ne les décrit**. Le modèle en trois phases, l'obligation d'un vault closed-ended pour créer un courtier, les bornes des dates : tout découvert par essai-erreur.

Aggravant : **côté serveur, toute violation de `VaultCreate` sort en `temMALFORMED` nu**, sans jamais nommer le champ. Les messages utiles — « `RedemptionDate` — `SubscriptionDate` must be within [180, 946708560) seconds » — n'existent **que dans la validation client de `xrpl.js`**. Un intégrateur en Python, en Java, ou qui signe à la main, n'a rien.

### Ce qu'on a vécu

Le cas qui nous a le plus coûté : tout `LoanSet` avec `GracePeriod` < 60 ou > `PaymentInterval` échoue en `temINVALID` « The transaction is ill-formed », sans indication de champ. **Une session de debug entière**, trois hypothèses fausses, puis un balayage systématique pour isoler la règle — qui n'est écrite nulle part.

| Envoyé                                           | Reçu                             | Signifiait réellement                      |
|--------------------------------------------------|----------------------------------|--------------------------------------------|
| <code>GracePeriod: 59</code>                     | <code>temINVALID</code>          | plancher non documenté à 60 s              |
| <code>LoanBrokerSet</code> sur vault open-ended  | <code>tecNO_PERMISSION</code>    | <code>V1_1</code> exige un closed-ended    |
| échéance proche de <code>RedemptionDate</code>   | <code>tecNO_PERMISSION</code>    | marge réelle ~90 s en temps ledger         |
| <code>StartDate</code> dans <code>LoanSet</code> | <code>disallowed location</code> | champ dérivé sur l'objet <code>Loan</code> |
| <code>Data</code> dans <code>LoanSet</code>      | <code>tesSUCCESS</code>          | <b>accepté puis silencieusement jeté</b>   |

## Ce qu'on propose

Publier la spec de V1\_1, même à l'état de brouillon — un amendement actif sur un réseau public devrait avoir une page. Et **remonter côté serveur les messages qui n'existent aujourd'hui que dans le SDK JavaScript** : c'est là qu'ils servent à tout le monde, quel que soit le langage.

## 5 · La co-signature LoanSet était invalide — corrigé en 5.2.0-beta.1

catégorie client libraries · lib xrpl.js 5.2.0-beta.0 → beta.1

### Le problème

signLoanSetByCounterparty signait avec le préfixe historique STX\0 alors que fixCleanup3\_4\_0, actif sur ce Devnet, exige CPT\0 en simple signature et CPM\0 en multisig. Le nœud refusait aux *local checks* : fails local checks: Counterparty: Invalid signature.

**Aucun LoanSet n'était soumettable depuis le SDK publié** — la transaction centrale de XLS-66, et l'exemple officiel createLoan.js avec elle. Les bons encodeurs existaient déjà dans ripple-binary-codec@2.11.0, avec l'amendement nommé en commentaire; ils n'étaient jamais appelés. Et .ci-config/xrpld.cfg s'arrêtant à fixCleanup3\_2\_0, la CI ne pouvait pas voir la casse.

### Ce qu'on a vécu

Deux heures à chercher une erreur de notre côté, jusqu'au moment où le même payload signé avec encodeForSigningCounterparty est passé. Sans prêt, un vault n'a ni rendement ni risque — **tout notre projet était bloqué** derrière cette signature. 5.2.0-beta.1 route désormais la signature par une table d'encodeurs indexée sur le rôle; revérifié sur la chaîne : tesSUCCESS, objet Loan créé.

### Ce qu'on propose

Deux points restent ouverts. **Ajouter fixCleanup3\_4\_0 à la configuration CI** de xrpl.js — sans ça la régression peut revenir sans qu'aucun test ne bronche, et c'est exactement ce qui a laissé passer le bug. Et **xrpl-py (counterparty\_signer.py) porte toujours le même défaut**, son codec ayant d'abord besoin des deux préfixes.

## 6 · Les MPT dans Escrow fonctionnent et ne sont documentés nulle part

catégorie documentation/tutorials · lib rippled 3.4.0-rc5

### Le problème

En cherchant un rail de règlement alternatif au Batch, on a testé les quatre candidats. Trois sont fermés et le disent clairement — CheckCreate → invalid SendMax, PaymentChannelCreate → Amount must be a string, OfferCreate → temDISABLED. Le quatrième marche de bout en bout, **et rien ne le dit** :

- EscrowCreate avec des parts de vault → tesSUCCESS, parts débitées à la création;
- EscrowFinish livre réellement, et crée l'objet MPToken du destinataire tout seul — là où un Payment exige un MPTokenAuthorize préalable;
- EscrowCancel restitue l'intégralité, et le contrôle de domaine s'applique **à la création comme au dénouement**;
- deux escrows partageant une même Condition forment un échange atomique **sans Batch** — mesuré,  $4 \times$  tesSUCCESS.

### Ce qu'on a vécu

Trouvé par hasard, en testant les rails un par un pour documenter ce qui est fermé. Capacité utile et invisible : la documentation d'Escrow ne mentionne pas les MPT, et rien dans XLS-33 ne renvoie vers Escrow.

**Un cas limite à signaler** : un escrow sans CancelAfter dont le destinataire perd son credential devient irrécupérable — EscrowFinish rend tecNO\_AUTH, EscrowCancel rend tecNO\_PERMISSION, les deux sorties sont fermées. Reproduit de bout en bout; nous le signalons sans savoir si c'est voulu.

### Ce qu'on propose

Documenter la compatibilité MPT × Escrow, et l'asymétrie d'autorisation : recevoir un MPT par Payment exige MPTokenAuthorize, par EscrowFinish non. C'est pour contourner la première que notre Batch compte **trois jambes au lieu de deux**.

### Ce qui nous a rassurés

**Personne ne peut enfermer un déposant** : les dates ne sont pas des champs de VaultSet, et un membre exclu d'un domaine peut toujours retirer. **Le pseudo-compte est étanche** — Payment, EscrowCreate, CheckCreate, PaymentChannelCreate → tecNO\_PERMISSION; vault dans un Batch → temINVALID\_INNER\_BATCH. **Aucune fuite d'arrondi** sur 145 aller-retours en XRP et en IOU. **Les BatchSigners couvrent les jambes internes** : un vendeur qui réécrit le prix après signature de l'acheteur est rejeté, et sa séquence interdit la double-vente. Enfin **tfAllOrNothing tient** dans tous les cas d'échec testés — ce qui manque n'est pas la garantie, c'est le moyen de la constater.

### Une question ouverte

OfferCreate avec un MPT répond temDISABLED : le code existe, l'amendement dort. **Le carnet d'ordres natif connaîtra-t-il les Permissioned Domains?** Nos parts sont gâtées; un carnet sans permission apparierait des ordres dont le règlement serait refusé. Selon la réponse, notre marché garde tout son sens — ou devient inutile.